



**Software Engineering Department**

**Capstone Project Phase A**

# **StudyNova: An AI-Powered Academic Planner**

**Github:** <https://github.com/halakadry/My-StudyNova>

**Advisor : Dr. Zeev Frenkel**

**Submitted by:**

**Hala Kadry 212698310**

**Adan Abu Ahmad 212802169**

# Table of Contents

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Abstract.....</b>                                     | <b>3</b>  |
| <b>1. Introduction.....</b>                              | <b>3</b>  |
| 1.1 The Problem We Want to Solve:.....                   | 3         |
| 1.2 Existing Solutions Today:.....                       | 3         |
| 1.3 Our Solution:.....                                   | 4         |
| 1.4 Stakeholders:.....                                   | 4         |
| 1.5 Book Structure:.....                                 | 5         |
| <b>2. Background.....</b>                                | <b>6</b>  |
| 2.1 Time Management and AI in Education.....             | 6         |
| 2.2 The most popular existing tools:.....                | 6         |
| 2.3 The Need for This Project:.....                      | 7         |
| <b>3. Expected Achievements.....</b>                     | <b>7</b>  |
| 3.1 Primary Objectives:.....                             | 7         |
| 3.2 Expected System Performance:.....                    | 8         |
| 3.3 Success Criteria:.....                               | 8         |
| <b>4. Engineering Process.....</b>                       | <b>9</b>  |
| 4.1 Process:.....                                        | 9         |
| 4.1.1 Motivation:.....                                   | 9         |
| 4.1.2 Research and Problem Analysis:.....                | 9         |
| 4.2 System Requirements:.....                            | 9         |
| 4.3 General Description & AI Algorithm:.....             | 10        |
| 4.3.1 AI-Based Optimization - Detailed Description:..... | 11        |
| 4.3.2 Input:.....                                        | 11        |
| 4.3.3 Output:.....                                       | 11        |
| 4.3.4 Compatibility with the system:.....                | 12        |
| 4.3.5 Algorithms Used:.....                              | 12        |
| 4.4 Technologies:.....                                   | 15        |
| 4.5 Dataset Description and Usage:.....                  | 15        |
| 4.6 Development Plan:.....                               | 16        |
| 4.7 Challenges and Constraints:.....                     | 17        |
| 4.8 Product:.....                                        | 18        |
| 4.8.1 Software Architecture :.....                       | 18        |
| 4.8.2 Use Case Diagram:.....                             | 19        |
| 4.8.3 Activity Diagram:.....                             | 21        |
| 4.8.4 User Interface:.....                               | 22        |
| <b>5. Verification and Evaluation.....</b>               | <b>24</b> |
| 5.1 Verification:.....                                   | 24        |
| 5.2 Evaluation:.....                                     | 25        |
| <b>6.References.....</b>                                 | <b>27</b> |

# **Abstract**

Managing academic time effectively is one of the most significant challenges faced by students today. Existing tools rely primarily on deadlines and fail to adapt to individual student behavior, task difficulty, or personal constraints. This leads to poor planning, work overload, and academic stress. This project presents StudyNova, an AI-powered web-based academic time management system built around four core features: AI-Powered Smart Scheduling, Dynamic Re-Scheduling, Overload Detection and Alerts, and a Performance Analytics Dashboard. The system is developed as a full-stack web application accessible from any device through a standard web browser.

## **1. Introduction**

### **1.1 The Problem We Want to Solve:**

Academic life places significant demands on students. Between attending lectures, completing assignments, preparing for exams, and managing personal responsibilities, students are constantly required to balance multiple competing priorities. Effective time management is therefore not just a useful skill – it is essential for academic success and personal well-being.

However, many students struggle to manage their time effectively [1]. Most rely on basic tools such as paper planners, phone reminders, or simple calendar apps that offer little more than deadline tracking. These tools do not account for important personal factors such as task difficulty, individual learning pace, energy levels, or daily constraints like commuting time and part-time work. As a result, students frequently experience work overload, poor planning, burnout, and unnecessary stress — all of which negatively impact their academic performance [2].

There is a clear need for a smarter, more adaptive solution, one that understands the student as an individual and helps them plan their time in a realistic, personalized, and intelligent way.

### **1.2 Existing Solutions Today:**

Several time management tools are currently available to students, including Google Calendar [3], Todoist [4], Notion [5], and MyStudyLife [6]. While these tools offer useful basic functionality such as task lists, reminders, and calendar views, they all share fundamental limitations when it comes to academic time management:

First, they are entirely deadline-driven; they remind students of when something is due but offer no guidance on how or when to work on it. Second, they do not adapt to the student's behavior or personal constraints. If a student misses a session or becomes overloaded, the tool does not respond or adjust. Third, none of these tools incorporates artificial intelligence to learn from the user's habits and improve recommendations over time. In summary, existing solutions treat all students the same way, offering a one-size-fits-all approach that fails to address the real complexity of student life.

### 1.3 Our Solution:

To address these limitations, we are developing an AI-Based Smart Academic Time Management System, a web application that combines modern development technologies with artificial intelligence-based tools to deliver a truly personalized planning experience. The system is built around four core unique features that distinguish it from any existing tool.

- **AI-Powered Smart Scheduling** automatically generates a personalized study plan based on each student's tasks, deadlines, difficulty levels, and personal habits.
- **Dynamic Re-Scheduling** ensures that whenever circumstances change, such as a missed session, a new task, or updated availability, the entire plan is automatically reorganized without any manual effort.
- **Overload Detection and Alerts** continuously monitor the student's workload and warn them before stress and burnout occur.
- **Performance Analytics Dashboard** provides visual insights into productivity trends and schedule adherence, helping students better understand and improve their habits over time.

Together, these features form a cohesive, intelligent system that does not just track tasks but actively supports the student throughout their academic journey. The system will be developed as a full-stack web application using **React.js** for the frontend, **Node.js** with **Express.js** for the backend, and **MongoDB** as the database. The AI scheduling and optimization module will be powered by the **Google Gemini API**, which will analyze student input and generate personalized study schedules. User authentication will be handled through **Firestore Authentication**, supporting both email/password and Google sign-in. The full system will be deployed on cloud platforms, making it accessible from any device through a standard web browser. A detailed description of the development process, technologies, and system architecture is provided in Chapter 4.

The scheduling algorithm is powered by the Google Gemini API. The system sends structured student data as input — including task titles, deadlines, difficulty levels, estimated durations, available study hours per day, and personal constraints — to the Gemini model. Gemini processes this information and returns an optimized study schedule in JSON format, specifying which task to study on each day and for how many hours. The backend then parses this output, stores the schedule in the database, and displays it to the student. Before sending data to Gemini, a priority scoring algorithm ranks tasks by urgency and difficulty to guide the AI toward the most important tasks. A detailed description of the algorithms is provided in Section 4.3.

### 1.4 Stakeholders:

The following groups have a direct interest in this system:

- **Students:** the primary users of the system. They benefit from reduced stress, better planning, and improved academic performance.
- **Academic Advisors and Lecturers:** benefit indirectly from students who are better prepared and more organized, leading to improved overall class performance.

- **Educational Institutions:** institutions such as colleges and universities benefit from tools that support student success and reduce dropout rates caused by academic overload.

## 1.5 Book Structure:

Our project book is organized into the following chapters covering various aspects of the project.

**Chapter 1: Introduction** presents the motivation behind the project, including the problem we aim to solve, a review of existing solutions and their limitations, our proposed solution with its four unique features, and the key stakeholders involved.

**Chapter 2: Literature Review** covers existing research and technologies related to academic time management, AI-based scheduling and optimization, and current tools available to students, concluding with an explanation of why a new solution is needed.

**Chapter 3: Expected Achievements** outlines the primary objectives of the project, the expected system performance, and the success criteria that will be used to evaluate whether the project has met its goals.

**Chapter 4: Engineering Process** is the most detailed chapter, covering the full development process from motivation and problem analysis through system requirements, architecture, technologies, development plan, and challenges. It also describes the final product, including software architecture, diagrams, and user interface design.

**Chapter 5: Verification and Evaluation** describes how the system will be tested and verified, and how its overall success will be evaluated against the defined criteria.

**Chapter 6: References** lists all academic sources, tools, and technologies referenced throughout this document.

## 2. Background

### 2.1 Time Management and AI in Education

Effective academic time management has been a widely studied topic in educational research. Studies have consistently shown that students who follow structured study plans tend to perform better academically and report lower levels of stress[12]. However, most traditional approaches to time management rely on manual scheduling and static tools that do not adapt to the student's individual behavior or changing circumstances.

Artificial intelligence has increasingly been applied in educational technology to personalize learning experiences. Machine learning algorithms can analyze user behavior patterns and make predictions about future activity, enabling systems to offer tailored recommendations. Techniques such as collaborative filtering, reinforcement learning, and constraint-based optimization have been explored for scheduling and task prioritization, showing promising results in adapting plans to individual needs.[7]

Scheduling optimization is also a well-studied problem in computer science. Algorithms such as genetic algorithms, greedy algorithms, and constraint satisfaction problems have been widely used to generate optimal schedules. In the context of academic planning, these techniques can be adapted to consider factors like task difficulty, deadlines, and personal availability, making them highly relevant to our project.[8]

### 2.2 The most popular existing tools:

Several time management applications are currently available to students. The most widely used include:

**Google Calendar** is one of the most popular scheduling tools available. It allows users to create events, set reminders, and share calendars. However, it is entirely manual – it does not suggest when to study or adapt to the user's workload.[3]

**Todoist** is a task management application that helps users organize tasks by priority and deadline. While it offers basic productivity features, it does not generate study plans or consider personal constraints such as commuting time or energy levels.[4]

**Notion** is a flexible productivity platform that combines note-taking, task management, and databases. Although highly customizable, it requires significant manual setup and offers no AI-driven scheduling or automatic plan adjustment.[5]

**MyStudyLife** is a student-focused planner that allows users to track classes, assignments, and exams. It is one of the closest tools to our system, but it still lacks AI-based behavior analysis, dynamic re-scheduling, and overload detection.[6]

All of these tools share a fundamental limitation: they are deadline-driven and passive. They remind students of what is due but offer no intelligent guidance on how to plan their time effectively. None of them adapt to the student's behavior, detect overload, or automatically reorganize the schedule when circumstances change.

## 2.3 The Need for This Project:

The review of existing research and tools makes it clear that there is a significant gap in the market. While tools like Google Calendar and MyStudyLife address basic scheduling needs, none of them offer a truly intelligent, adaptive, and student-centered solution.

During our studies, we noticed that many students rely on reminders, notebooks, or digital calendars to keep track of their academic responsibilities. While these tools help record deadlines, they do not help students decide how to distribute their study time effectively. As a result, many students postpone difficult tasks, underestimate the time required to complete assignments, or become overwhelmed when several deadlines occur close together. These observations highlighted the need for a smarter and more adaptive planning solution.

This project directly addresses that gap. By integrating AI-powered smart scheduling, dynamic re-scheduling, overload detection, and a performance analytics dashboard into one cohesive system, we aim to deliver a solution that genuinely supports students in managing their academic lives more effectively. The need for such a system is supported both by academic research on student stress and time management and by the clear limitations of the tools currently available.

## 3. Expected Achievements

### 3.1 Primary Objectives:

The primary objective of this project is to design and develop a fully functional, AI-based web application that helps academic students manage their time more effectively. The following objectives define the core goals of the system:

- **Develop a complete full-stack web application** that is accessible from any device through a web browser, with a clean and intuitive user interface.
- **Implement AI-powered smart scheduling** that automatically generates personalized study plans based on each student's tasks, deadlines, difficulty levels, and personal habits.
- **Implement dynamic re-scheduling** that automatically reorganizes the student's plan whenever circumstances change, such as a missed session or a newly added task.
- **Implement overload detection and alerts** that continuously monitor the student's workload and notify them when they are at risk of burnout or overcommitment.
- **Implement a performance analytics dashboard** that provides students with visual insights into their productivity, progress, and schedule adherence over time.
- **Deliver a system that is practical and usable** by real students, solving a genuine problem in academic life.

### 3.2 Expected System Performance:

Upon completion, the system is expected to perform as follows:

- The AI scheduling module will generate a realistic and personalized study plan within seconds of the student inputting their tasks and constraints.
- The dynamic re-scheduling feature will automatically update the plan whenever the student manually marks a task as incomplete or requests a plan update. The system will then reorganize the remaining schedule accordingly.
- The overload detection system will analyze the student's current workload based on their input tasks and deadlines, and alert them when the system detects a high-risk period. Students can also manually report their stress level or difficulty, which the system will use to further adjust the plan.
- The performance analytics dashboard will display up-to-date statistics and trends based on the student's recorded activity, updating as the student logs their progress.
- The system will be responsive and fast, handling multiple users simultaneously without performance degradation.
- The user interface will be simple and intuitive enough for any student to use without prior training or technical knowledge.

### 3.3 Success Criteria:

The project will be considered successful if the following criteria are met:

- **Functional completeness** - all core features (AI scheduling, dynamic re-scheduling, overload detection, and analytics dashboard) are fully implemented and working correctly.
- **Usability** - real students are able to use the system easily and find it helpful in managing their academic workload.
- **Accuracy** - the AI scheduling module generates realistic and achievable study plans that respect the student's constraints and priorities. Accuracy will be measured by the following criteria:
  1. The generated schedule must not assign study sessions that exceed the student's available hours per day .this will be verified automatically by the system for every generated schedule.
  2. The overload detection algorithm must achieve 100% accuracy in detecting days where the total assigned study hours exceed the student's available hours, as this is a direct algorithmic calculation with no ambiguity.
- **Reliability** - the system performs consistently without critical bugs or crashes during normal use.
- **Relevance** - the system addresses the core problem defined in Chapter 1, helping students reduce stress and improve their time management.

## 4. Engineering Process

### 4.1 Process:

This section describes the steps we followed throughout the development of the project, from the initial motivation and research phase through to the final system design and implementation plan.

#### 4.1.1 Motivation:

The idea for StudyNova originated from our own experience as engineering students. Throughout our studies, we often found ourselves facing multiple assignments, laboratory reports, exams, and personal commitments at the same time. Although we used tools such as Google Calendar and task management applications, we still struggled to decide when to study, which tasks to prioritize, and how to adjust our plans when unexpected changes occurred. These experiences led us to believe that students need more than a simple calendar or reminder system. They need an intelligent assistant that can help them organize their academic workload in a realistic and personalized way. This motivated us to develop StudyNova.

#### 4.1.2 Research and Problem Analysis:

Before beginning development, we conducted research to better understand the problem and the existing landscape of solutions. We reviewed academic studies on student time management and stress, analyzed existing tools such as Google Calendar, Todoist, Notion, and MyStudyLife, and identified their key limitations.

Our research confirmed that the core problem is not a lack of tools, but a lack of intelligent tools. Students need a system that goes beyond deadline tracking and actively helps them plan, adapt, and improve. We also identified the key factors that a smart scheduling system must consider, including task difficulty, estimated completion time, personal constraints, and the student's own behavior patterns.

Based on this analysis, we defined the scope and direction of our project, focusing on four core unique features: AI-powered smart scheduling, dynamic re-scheduling, overload detection and alerts, and a performance analytics dashboard.

### 4.2 System Requirements:

#### Functional Requirements

- **User Registration and Login** - students can create an account and securely log in to access their personal dashboard.
- **Task and Exam Input** - students can add academic tasks and exams with details such as title, deadline, estimated duration, and difficulty level.
- **AI-Powered Schedule Generation** - the system automatically generates a personalized study schedule based on the student's inputted tasks, deadlines, difficulty levels, and available time slots.
- **Task Completion Marking** - students can manually mark tasks as complete or incomplete, allowing the system to track progress accurately.

- **Re-Scheduling on Request** - when a student marks a task as incomplete or requests a plan update, the system automatically reorganizes the remaining schedule.
- **Overload Detection and Alerts** - the system analyzes the student's workload and alerts them when a high-risk overload period is detected, suggesting adjustments.
- **Stress and Difficulty Reporting** - students can manually report their current stress level or task difficulty, which the system uses to further personalize the schedule.
- **Performance Analytics Dashboard** - the system displays visual statistics on productivity, task completion rates, and schedule adherence over time.
- **Profile Management** - students can update their personal information and constraints such as available study hours and daily commitments.

## Non-Functional Requirements

- **Usability** -the interface must be clean, intuitive, and easy to use without any prior training or technical knowledge.
- **Performance** - the system must respond quickly to user actions and generate schedules within a few seconds.
- **Scalability** -the system must be able to handle multiple users simultaneously without performance degradation.
- **Security:**
  1. All user data must be stored securely, and authentication must be properly implemented to protect user accounts.
  2. If a user enters an incorrect password 3 times consecutively, the account will be temporarily locked and the user will not be able to try again for 15 minutes.
  3. Password reset will be available via a verification email sent to the student's registered email address.
  4. Users can register and log in using their email and password, or sign in using their Google account
- **Reliability** -the system must operate consistently without critical bugs or crashes during normal use.
- **Accessibility** - the system must be accessible from any device through a standard web browser, without requiring any installation.
- **Maintainability** - the codebase must be well-structured and documented to allow future improvements and bug fixes.

## 4.3 General Description & AI Algorithm:

StudyNova is a web-based application built around four core features that distinguish it from existing tools.

The first is **AI-Powered Smart Scheduling**, which automatically creates a realistic study plan for the student. The second is **Dynamic Re-Scheduling**, which reorganizes the plan whenever the student marks a task as incomplete or requests an update. The third is

**Overload Detection and Alerts**, which monitors the student's workload and notifies them when they are at risk of being overloaded. The fourth is a **Performance Analytics Dashboard**, which provides the student with visual insights into their productivity and progress over time.

The system consists of three main components: a frontend interface that the student interacts with, a backend server that handles the business logic and AI processing, and a database that stores all user data, tasks, and schedules. The application is accessible from any device through a standard web browser, requiring no installation.

### 4.3.1 AI-Based Optimization - Detailed Description:

The AI scheduling module is the core component of StudyNova. It is powered by the Google Gemini API (**Gemini-1.5-flash** model), which receives student data from the backend and returns an optimized study schedule.

### 4.3.2 Input:

The backend sends a structured JSON object to the Gemini API containing the following information(see Figure 1) :

- **List of tasks:** title, deadline, difficulty level, and estimated duration in hours
- **List of exams:** subject name and exam date
- **Student's available study hours per day**
- **Personal constraints** such as commuting time or work hours
- **Current stress level** is manually reported by the student

**Note:**Difficulty level is self-reported by the student when adding a task. Since this is subjective and varies between students, the system also uses the Google Gemini API to analyze the task description and automatically suggest an appropriate difficulty level.

```
{
  "tasks": [
    {"title": "Data Structures Assignment", "deadline": "2025-05-25", "difficulty": "hard", "duration_hours": 4},
    {"title": "Algorithms Exam Preparation", "deadline": "2025-05-28", "difficulty": "hard", "duration_hours": 6},
    {"title": "OS Lab Report", "deadline": "2025-05-30", "difficulty": "easy", "duration_hours": 2}
  ],
  "exams": [
    {"subject": "Algorithms", "date": "2025-05-28"}
  ],
  "available_hours_per_day": 4,
  "constraints": {"commuting_hours": 1},
  "stress_level": "medium"
}
```

**Figure 1:** Example of structured JSON input sent to the Google Gemini API

### 4.3.3 Output:

The Gemini API returns a structured JSON schedule specifying which task to study on each day and for how many hours. The schedule is designed to respect the student's available time, prioritize tasks by deadline and difficulty, and distribute the workload evenly to avoid overload.

```

{
  "schedule": [
    {"date": "2025-05-22", "task": "Data Structures Assignment", "hours": 2},
    {"date": "2025-05-23", "task": "Algorithms Exam Preparation", "hours": 3},
    {"date": "2025-05-24", "task": "Data Structures Assignment", "hours": 2},
    {"date": "2025-05-25", "task": "Algorithms Exam Preparation", "hours": 3},
    {"date": "2025-05-26", "task": "OS Lab Report", "hours": 2}
  ]
}

```

**Figure 2:** Example of structured JSON output returned by the Google Gemini API

#### 4.3.4 Compatibility with the system:

The JSON output is received and parsed by the Node.js backend. The parsed schedule is then stored in MongoDB and retrieved by the React.js frontend, which displays it to the student in the weekly calendar view. If the generated schedule contains overloaded days, where the total study hours exceed the student's available time, the overload detection module flags those days and sends an alert to the student, suggesting adjustments.

#### 4.3.5 Algorithms Used:

##### 1. Large Language Model (LLM)

The core AI scheduling is performed by Google Gemini. Rather than using a fixed scheduling formula, the system constructs a structured natural language prompt that is sent to the Gemini API. The prompt is built by the backend using the following algorithm:

##### Prompt Construction Algorithm:

```

1. Collect all student tasks with their titles, deadlines,
   difficulty levels, and estimated durations
2. Collect all exam dates and subjects
3. Calculate the number of available study days until
   the nearest deadline
4. Apply the priority scoring algorithm to rank tasks
5. Build a structured prompt in the following format:
   - Specify the student's available hours per day
   - List all tasks ordered by priority score
   - List all exam dates
   - Specify personal constraints
   - Instruct Gemini to return a JSON schedule

```

**Figure 3:** Prompt Construction Algorithm , Steps for building the structured prompt sent to the Gemini AI API

## Prompt Construction Example

### Prompt sent to Gemini:

You are a study planner. Create a study schedule for a student with the following tasks and constraints. Return the result as a JSON object only, with no additional text.

Available study hours per day: 4 hours

Constraints: 1 hour commuting daily

Tasks (ordered by priority):

- Data Structures Assignment | Deadline: 2025-05-25 | Difficulty: Hard | Duration: 4 hours
- Algorithms Exam Preparation | Deadline: 2025-05-28 | Difficulty: Hard | Duration: 6 hours
- OS Lab Report | Deadline: 2025-05-30 | Difficulty: Easy | Duration: 2 hours

Exams:

- Algorithms Exam: 2025-05-28

Generate a day-by-day study schedule from today until all tasks are completed. Do not assign more than 4 hours per day. Prioritize tasks with earlier deadlines.

### Example Output from Gemini:

```
{
  "schedule": [
    {
      "date": "2025-05-22",
      "task": "Data Structures Assignment",
      "hours": 2
    },
    {
      "date": "2025-05-23",
      "task": "Algorithms Exam Preparation",
      "hours": 3
    },
    {
      "date": "2025-05-24",
      "task": "Data Structures Assignment",
      "hours": 2
    },
    {
      "date": "2025-05-25",
      "task": "Algorithms Exam Preparation",
      "hours": 3
    },
    {
      "date": "2025-05-26",
      "task": "OS Lab Report",
      "hours": 2
    }
  ]
}
```

**Figure 4:** Example of structured JSON schedule returned by the Google Gemini API"

## 2. Priority Scoring Algorithm

Before sending data to Gemini, the backend applies a priority scoring algorithm to rank tasks by urgency and importance. Each task receives a priority score calculated as follows:

$$\text{Priority Score} = (1 / \text{days\_until\_deadline}) * \text{difficulty\_weight} * \text{duration\_weight}$$

Where:

- **days\_until\_deadline** - number of days remaining before the task deadline
- **difficulty\_weight** - Easy = 1, Medium = 1.5, Hard = 2

- **duration\_weight** - estimated hours divided by available daily hours

### Worked Example:

Using the sample task "Data Structures Assignment" from Figure 1 (deadline: 2025-05-25, difficulty: Hard, duration: 4 hours), with a start date of 2025-05-22 and available study hours per day of 4:

- days\_until\_deadline = 3 (from 2025-05-22 to 2025-05-25)
- difficulty\_weight = 2 (Hard)
- duration\_weight =  $4 \div 4 = 1$

$$\text{Priority Score} = (1 / 3) \times 2 \times 1 = 0.67$$

For comparison, the "OS Lab Report" task (deadline: 2025-05-30, difficulty: Easy, duration: 2 hours):

- days\_until\_deadline = 8
- difficulty\_weight = 1 (Easy)
- duration\_weight =  $2 \div 4 = 0.5$

$$\text{Priority Score} = (1 / 8) \times 1 \times 0.5 = 0.0625$$

Since 0.67 is significantly higher than 0.0625, "Data Structures Assignment" is ranked as higher priority and is scheduled earlier — consistent with the output shown in Figure 2, where Data Structures Assignment appears first in the schedule.

Tasks with higher scores are flagged as high priority and passed to Gemini with instructions to schedule them earlier.

### 3. Overload Detection Algorithm

After the schedule is generated, the backend runs an overload detection check on each day:

```
For each day in schedule:
    total_hours = sum of study hours assigned to that day
    if total_hours > available_hours_per_day:
        flag day as OVERLOADED
        send alert to student
```

**Figure 5:** Overload Detection pseudocode showing how each day is flagged when study hours exceed availability

If more than 30% of scheduled days are flagged as overloaded, the system automatically requests a new schedule from Gemini. The retry prompt explicitly identifies which days were overloaded and by how many hours, instructing Gemini to redistribute the excess study hours from those days to days with lighter workloads, while still respecting all task deadlines. If the regenerated schedule still fails the quality check after 3 retry attempts, the system stops retrying and alerts the student directly, so they can manually adjust their tasks or deadlines instead of receiving an unreliable plan.

## 4.4 Technologies:

**Frontend - React.js :** The user interface will be developed using React.js, a widely used JavaScript library for building dynamic and responsive web applications. React allows the development of reusable UI components, making it suitable for building an interactive dashboard and schedule view. Its large community and extensive documentation make it an appropriate choice for a student development project.[10]

**Backend -Node.js with Express.js:** The server-side logic will be implemented using Node.js with the Express.js framework. Node.js provides a fast and scalable runtime environment for handling API requests, while Express.js simplifies the creation of RESTful APIs. This combination is well-suited for managing communication between the frontend, database, and AI module.

**Database - MongoDB:** Student data, tasks, schedules, and behavior logs will be stored in MongoDB, a NoSQL database that stores data in a flexible, JSON-like format. MongoDB is particularly suitable for this project due to the dynamic and variable nature of student task data, which does not always fit a rigid relational structure.[11]

**AI Module - Google Gemini API:** The AI scheduling and optimization module will be powered by the Google Gemini API, a free and powerful large language model developed by Google. Gemini will be used to analyze student input such as tasks, deadlines, difficulty levels, and available time slots, and generate a personalized, optimized study schedule. The API will be integrated into the backend server, allowing seamless communication between the user interface and the AI module. Google Gemini was selected for its free tier availability, ease of integration, and strong natural language understanding capabilities.

**Authentication — Firebase Authentication:** User authentication will be handled using Firebase Authentication, which supports both email/password login and Google sign-in. This simplifies the implementation of secure authentication without requiring a custom authentication system.[9]

**Deployment:** The frontend will be deployed using Vercel, and the backend will be hosted on Render. Both platforms offer free tiers suitable for a student project and support continuous deployment from GitHub.

## 4.5 Dataset Description and Usage:

Unlike research-based projects that rely on pre-existing datasets, StudyNova is a development project that generates and uses data dynamically through user interaction. The system does not require an external dataset to function. Instead, all data is collected directly from the students who use the system.

The data used by the system consists of two main types:

**User-Inputted Data:** Each student provides the system with personal academic data, including:

- Task and assignment details (title, deadline, estimated duration, difficulty level)
- Exam dates and subjects
- Available study hours per day
- Personal constraints such as commuting time or work hours

**System-Generated Data:** As students interact with the system over time, the following behavioral data is recorded:

- Task completion rates and timestamps
- Schedule adherence (whether the student followed the generated plan)
- Stress and difficulty reports submitted manually by the student
- Re-scheduling requests and frequency

This data is stored securely in the MongoDB database and is used by the Google Gemini API to generate personalized, optimized study schedules. The more data a student provides, the more accurate and personalized the generated schedule becomes. All data is private and accessible only to the individual student.

## 4.6 Development Plan:

The development of StudyNova will be carried out in a structured and incremental manner, divided into the following phases:

### Phase 1 — Project Setup and Planning (Weeks 1-2)

- Set up the development environment and project repository on GitHub
- Define the system architecture and database schema
- Configure the frontend (React.js), backend (Node.js), and database (MongoDB)
- Set up Firebase Authentication

### Phase 2 — Core Backend Development (Weeks 3-5)

- Implement user registration and login functionality
- Develop REST API endpoints for task and exam management
- Implement database models for users, tasks, schedules, and behavior logs
- Integrate Firebase Authentication with the backend

### Phase 3 — AI Module Integration (Weeks 6-7)

- Integrate the Google Gemini API into the backend
- Develop the scheduling logic that sends student data to Gemini and processes the response
- Implement the overload detection algorithm
- Test the AI module with sample student data

### Phase 4 — Frontend Development (Weeks 8-10)

- Develop the student dashboard and task input forms
- Implement the schedule view and calendar interface
- Develop the performance analytics dashboard with visual charts
- Connect the frontend to the backend REST API

## **Phase 5 — Integration and Testing (Weeks 11-12)**

- Integrate all system components together
- Perform unit testing, integration testing, and user testing
- Fix bugs and refine the user interface based on feedback
- Verify that all success criteria are met

## **Phase 6 — Deployment and Final Submission (Weeks 13-14)**

- Deploy the frontend on Vercel and the backend on Render
- Perform final system testing on the deployed version
- Prepare the project book and presentation
- Submit the final deliverables

## **4.7 Challenges and Constraints:**

During the development of StudyNova, we anticipate several challenges and constraints that may affect the project. The following describes each challenge and how we plan to address it.

**1. AI Scheduling Quality** One of the main challenges is ensuring that the Google Gemini API generates realistic and useful study schedules. The quality of the output depends heavily on how well we design the prompts and structure the student data sent to the API. Poorly designed prompts may result in generic or impractical schedules. To address this, we will invest time in prompt engineering and test the AI module extensively with different types of student data.

**2. Data Privacy and Security** Since the system handles personal academic data, ensuring data privacy is a critical constraint. Students must trust that their data is stored securely and not accessible to others. To address this, we will implement proper authentication using Firebase, encrypt sensitive data, and follow best practices for secure API development.

**3. Dynamic Re-Scheduling Complexity** Implementing a dynamic re-scheduling feature that automatically reorganizes the entire plan when a change occurs is technically complex. The system must handle multiple interdependent tasks and constraints simultaneously. We will address this by carefully designing the scheduling algorithm and testing it with various scenarios.

**4. User Adoption and Usability** Even a technically strong system will fail if students find it difficult to use. Designing an interface that is intuitive and appealing to students is a significant challenge. We will address this by following established UI/UX design principles and conducting user testing with real students to gather feedback.

**5. Time and Resource Constraints** As a student project with a fixed deadline, time is a significant constraint. Balancing the development of multiple complex features within the available time requires careful planning. We will address this by following the development plan outlined in Section Development Plan (Section 4.6) and prioritizing core features over optional enhancements.

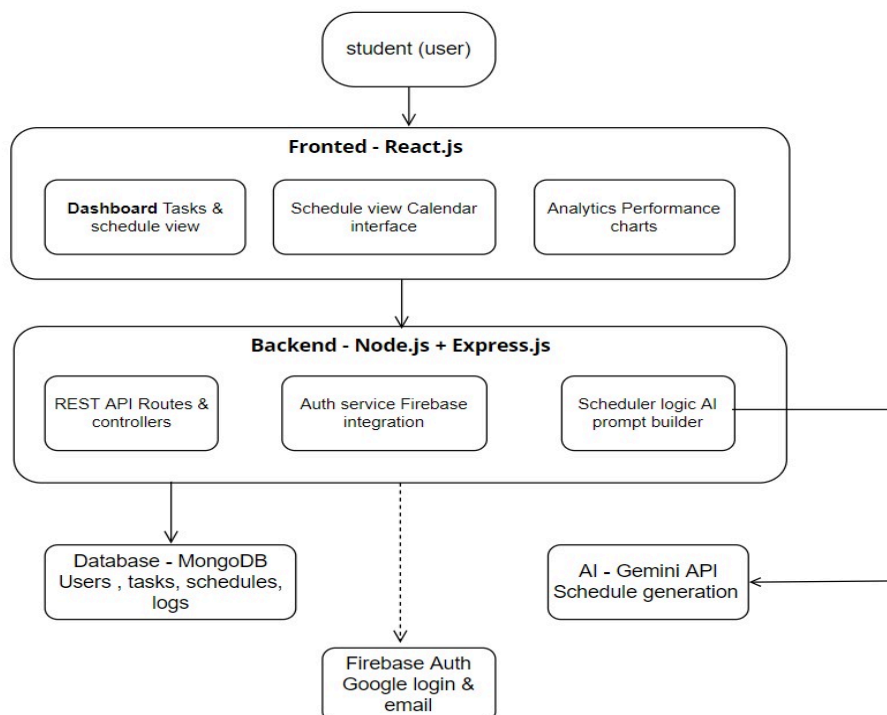
**6. Integration Between Components** Integrating the frontend, backend, database, and AI module into one cohesive system can be challenging, especially when each component is

developed separately. We will address this by defining clear API contracts between components from the beginning and performing regular integration testing throughout the development process.

## 4.8 Product:

This section describes the final product of the StudyNova project — the actual system that will be built. It includes the software architecture, UML diagrams, and user interface design. Together, these elements provide a complete and detailed picture of how the system is structured, how it operates, and how students will interact with it.

### 4.8.1 Software Architecture :



StudyNova follows a three-layer client-server architecture consisting of a frontend, a backend, and a database, with an external AI module integrated into the backend layer.

**Frontend Layer - React.js** The frontend is built using React.js and serves as the main interface through which students interact with the system. It consists of three main views: the Dashboard where students input tasks and exams, the Schedule View, which displays the generated study plan in a calendar format, and the Analytics View, which presents visual performance statistics.

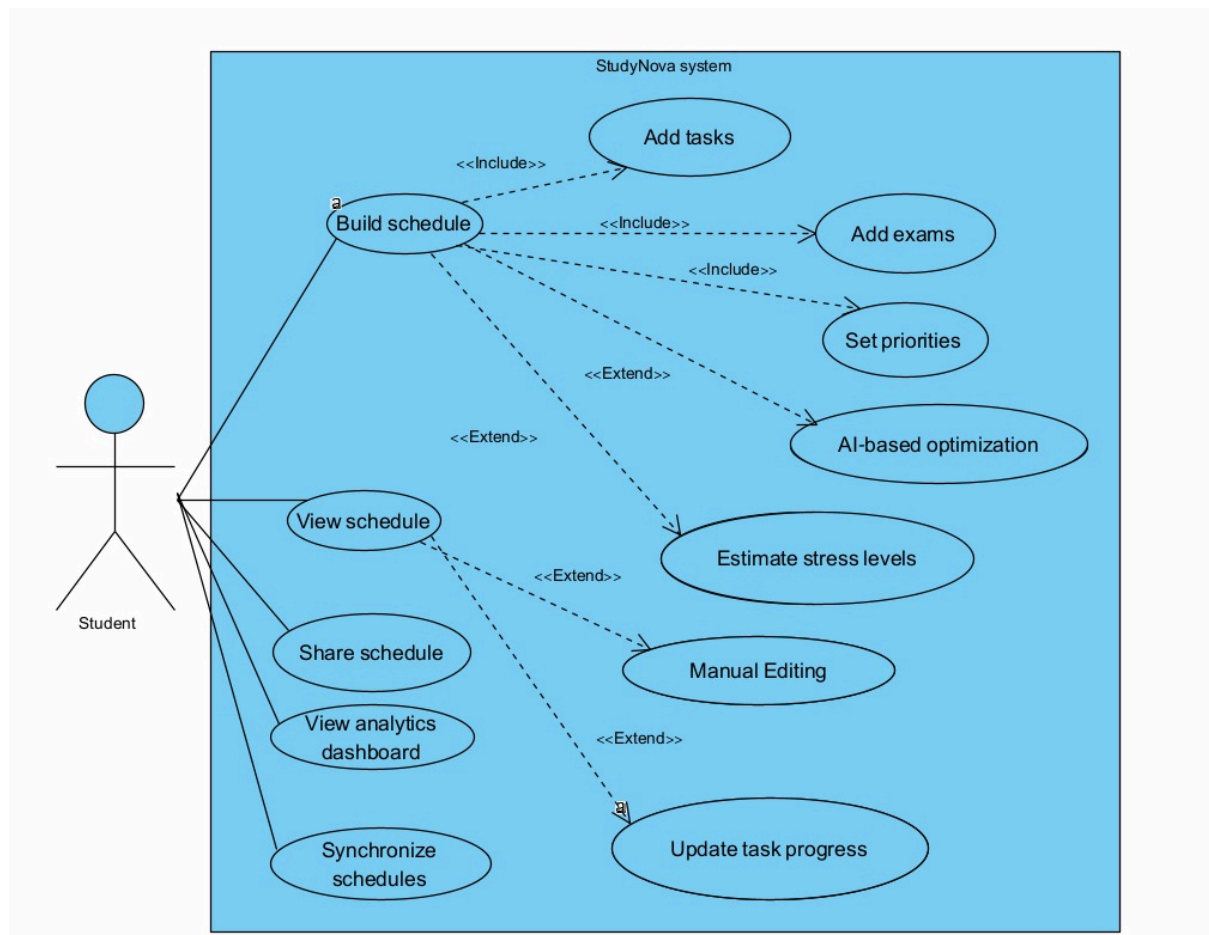
**Backend Layer -Node.js + Express.js** The backend handles all business logic and serves as the communication hub between the frontend, database, and AI module. It consists of three main components: the REST API, which handles all incoming requests from the frontend, the Authentication Service, which integrates with Firebase to manage user login, and the Scheduler Logic, which builds and sends prompts to the Google Gemini API and processes the responses.

**Database - MongoDB** MongoDB stores all persistent data, including user profiles, tasks, exams, generated schedules, and behavior logs. Its flexible document structure is well-suited for the dynamic nature of student data.

**AI Module - Google Gemini API** The AI module is powered by the Google Gemini API. When a student requests a schedule, the backend sends the student's tasks, deadlines, difficulty levels, and constraints to Gemini, which returns an optimized study plan that is then stored and displayed to the student.

**Authentication - Firebase Authentication** User authentication is handled externally through Firebase Authentication, supporting both email/password login and Google sign-in.

#### 4.8.2 Use Case Diagram:



The use case diagram above presents the main interactions between the student (the primary actor) and the StudyNova system. The student can perform the following actions:

**1. Build Schedule** - the main use case, which includes three sub-actions:

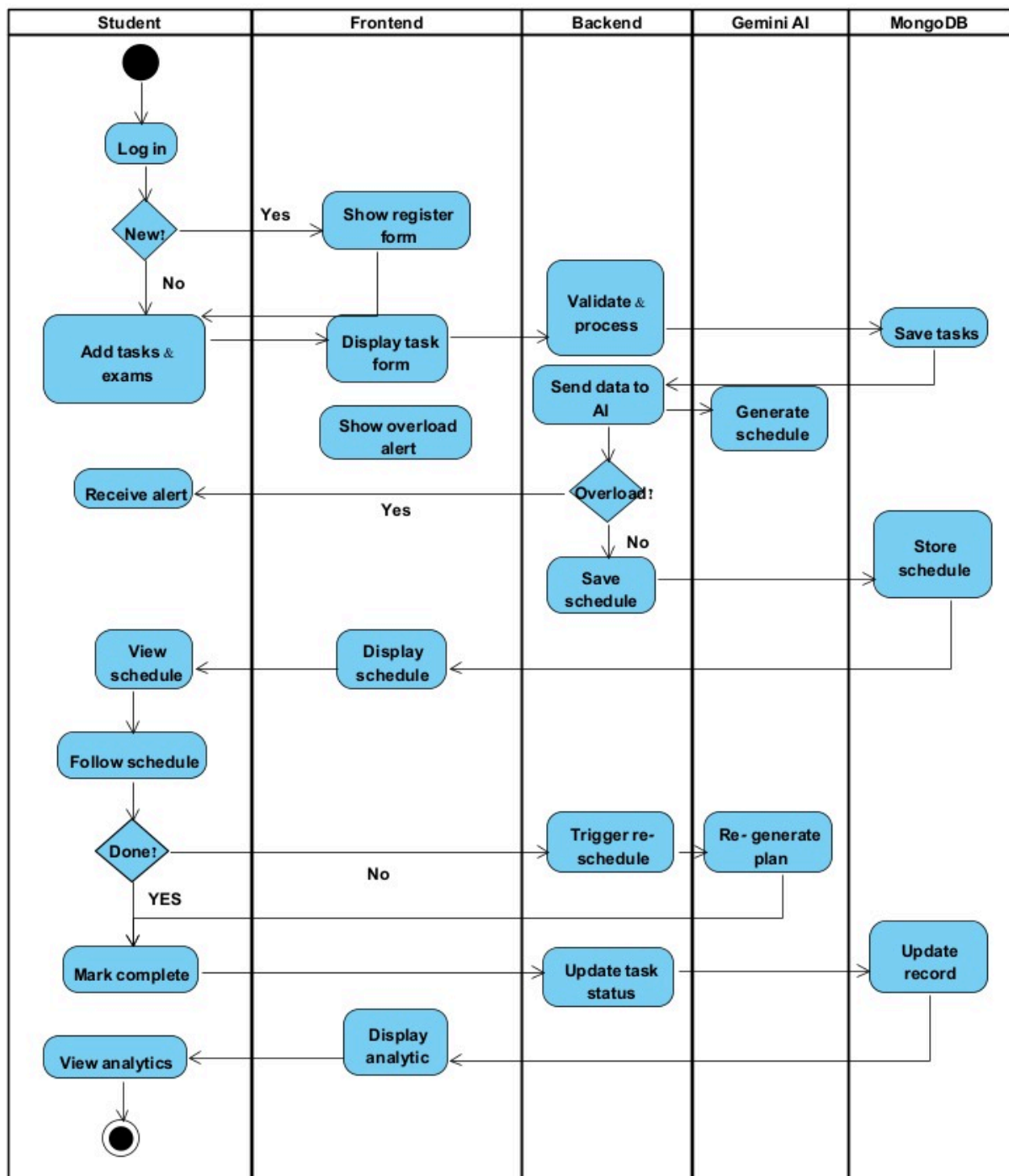
- **Add tasks** - input academic tasks and assignments with details such as title, deadline, difficulty level, and estimated duration
- **Add exams** - input exam dates and subjects
- **Set priorities** - define the importance and urgency of tasks to help the AI generate a better schedule

This use case is extended by two AI-driven actions:

- **AI-based optimization** - the system automatically analyzes the schedule and suggests improvements based on the student's constraints and priorities

- **Estimate stress levels** - the system estimates the student's workload and stress level based on the current schedule
2. **View Schedule** - the student views the AI-generated study plan. This use case is extended by:
- **Manual editing** - the student can manually adjust the generated schedule
  - **Update task progress** - the student updates the completion status of tasks
3. **Share Schedule** - the student can share their study schedule with other users such as classmates or study partners.
4. **Synchronize Schedules** - the student can synchronize their schedule with other users to coordinate shared study sessions and avoid conflicts.
5. **View Analytical Dashboard** - the student views visual statistics about their productivity, task completion rates, and schedule adherence over time.

### 4.8.3 Activity Diagram:



The activity diagram above illustrates the full flow of the StudyNova system across five swimlanes: Student, Frontend, Backend, Gemini AI, and MongoDB.

**Login and Registration:** The flow begins when the student logs in. If they are a new user, the frontend displays a registration form, and the account is created. Otherwise, the student proceeds directly to the next step.

**Adding Tasks and Exams:** The student inputs their tasks, exams, and personal constraints through the frontend form. The backend validates and processes the data, which is then saved to MongoDB.

**AI Schedule Generation:** The backend sends the student's data to the Google Gemini API, which generates a personalized, optimized study schedule. The backend then checks whether the generated schedule causes an overload situation. If overload is detected, the frontend displays an alert to the student. The generated schedule is saved to MongoDB and displayed to the student through the frontend.

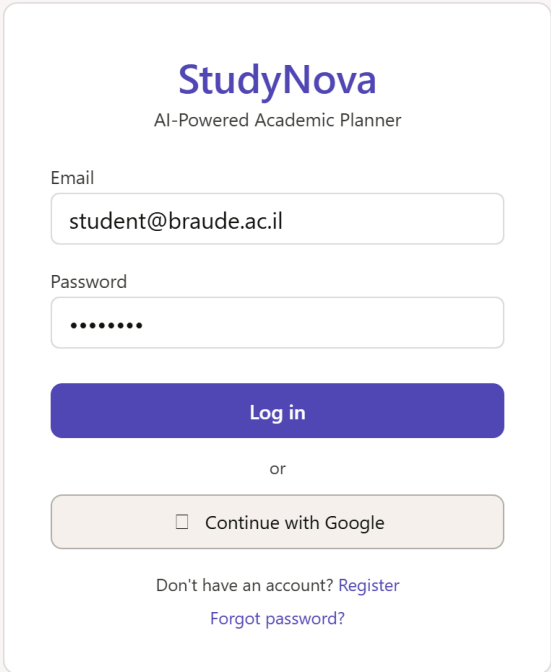
**Following the Schedule:** The student follows the generated schedule. For each task, the system checks whether it was completed. If a task is not completed, the backend triggers a re-scheduling process — the Gemini AI generates a new plan, which is sent back through the backend and frontend to the student. If the task is completed, the student marks it as done, and the backend updates the task status in MongoDB.

**Analytics** After completing tasks, the frontend displays the performance analytics dashboard, allowing the student to view their productivity trends and progress. The flow then ends.

#### 4.8.4 User Interface:

The StudyNova user interface is designed to be clean, intuitive, and accessible from any web browser. It consists of four main screens:

##### Screen 1 — Login Page :



StudyNova  
AI-Powered Academic Planner

Email  
student@braude.ac.il

Password  
.....

Log in

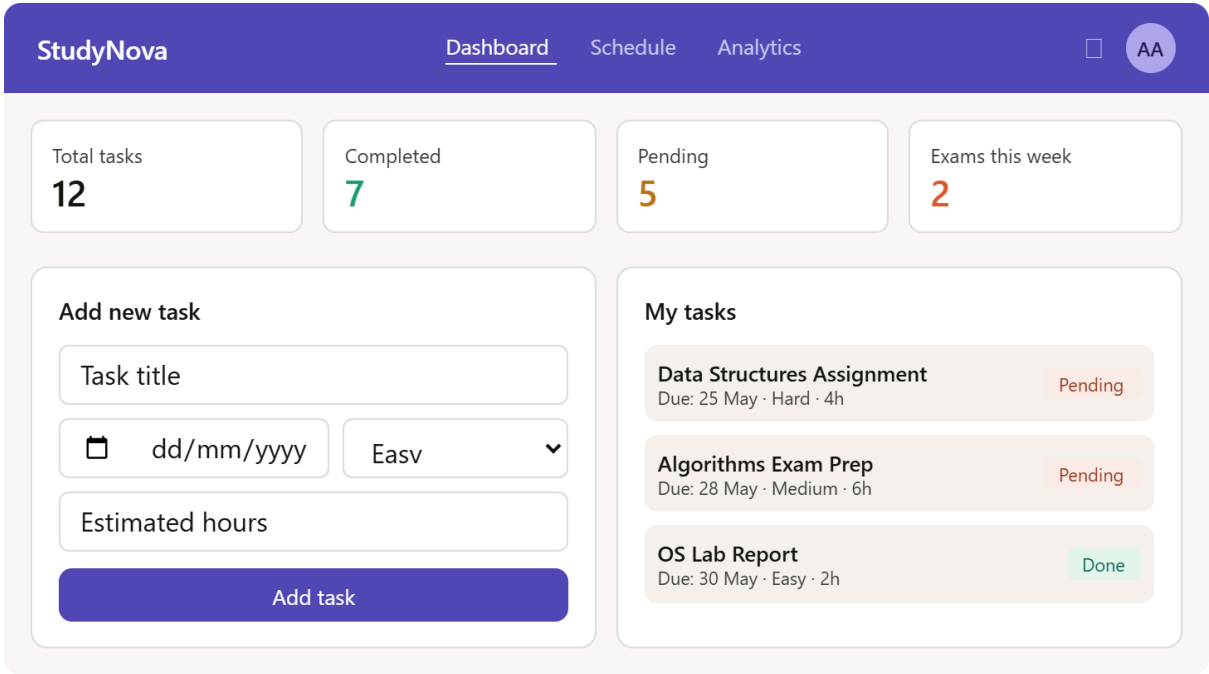
or

☐ Continue with Google

Don't have an account? [Register](#)  
[Forgot password?](#)

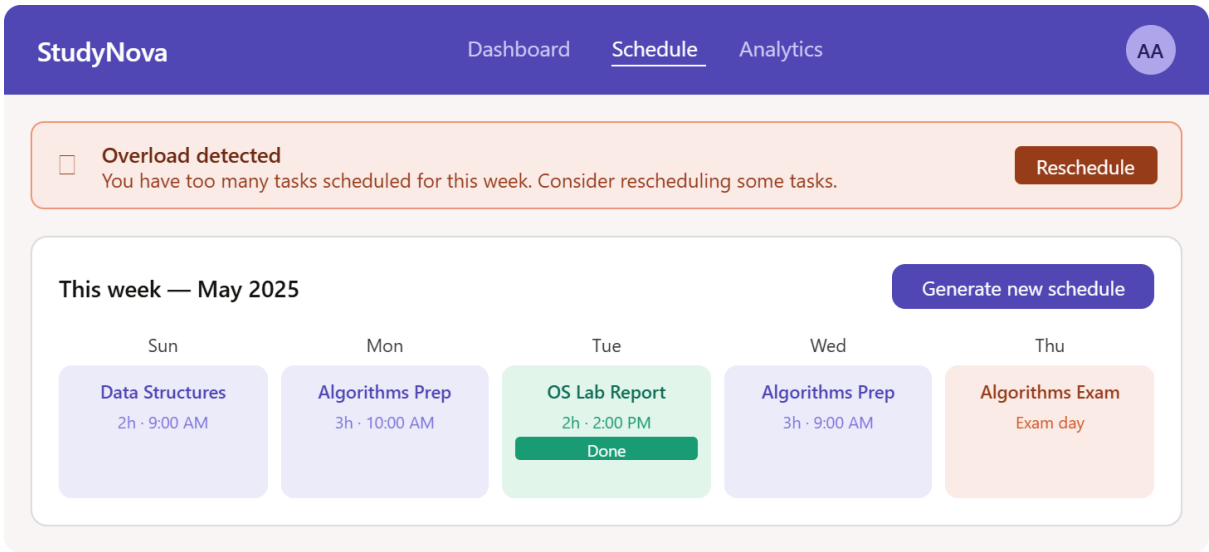
The login page allows students to sign in using their email and password or through their Google account. New students can register directly from this page, and a forgotten password option is available for account recovery.

Screen 2 — Dashboard :



The dashboard is the main screen of the application. It displays four summary cards showing total tasks, completed tasks, pending tasks, and upcoming exams. Students can add new tasks by entering the task title, deadline, difficulty level, and estimated duration. All existing tasks are listed with their status (pending or done).

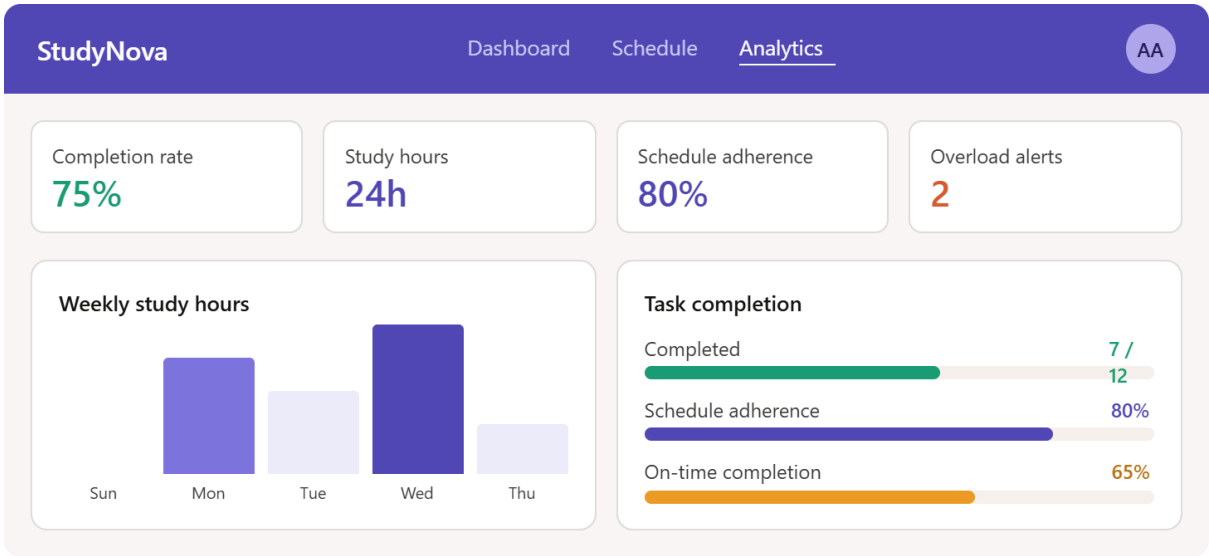
Screen 3 — Schedule View :



The schedule view displays the AI-generated study plan in a weekly calendar format. Each day shows the assigned study sessions with the task name, duration, and time. When an

overload situation is detected, a warning alert is shown at the top of the screen with an option to request automatic re-scheduling.

Screen 4 — Analytics Dashboard :



The analytics screen provides students with visual insights into their performance. It shows four key metrics: completion rate, total study hours, schedule adherence, and number of overload alerts. A bar chart displays daily study hours for the week, and progress bars show task completion rate, schedule adherence, and on-time completion percentages.

5. Verification and Evaluation

5.1 Verification:

Verification is the process of ensuring that the system has been built correctly and that all features work as intended. The following tests will be performed to verify the StudyNova system:

Functional Testing:

Each functional requirement will be tested individually to confirm that it works correctly:

| Test               | Description                                                 | Expected Result                               |
|--------------------|-------------------------------------------------------------|-----------------------------------------------|
| User registration  | Register a new account with an email and a password         | Account created successfully                  |
| Google login       | Sign in using Google account                                | User logged in successfully                   |
| Add task           | Add a task with a title, deadline, difficulty, and duration | Task saved and displayed in the task list     |
| Generate schedule  | Request AI schedule generation                              | Personalized schedule generated and displayed |
| Mark task complete | Mark a task as done                                         | Task status updated to                        |

|                     |                                                |                                         |
|---------------------|------------------------------------------------|-----------------------------------------|
|                     |                                                | completed                               |
| Re-scheduling       | Mark the task incomplete and request an update | Schedule automatically reorganized      |
| Overload detection  | Adding too many tasks in a short period        | Overload alert displayed to the student |
| Analytics dashboard | View performance statistics                    | Correct statistics displayed            |
| Password lock       | Enter the wrong password 3 times               | Account locked for 15 minutes           |
| Password reset      | Request a password reset via email             | Reset email sent successfully           |

**Unit Testing:** Each component of the system will be tested independently:

- Frontend components will be tested to ensure correct rendering and user interactions
- Backend API endpoints will be tested to ensure correct responses for valid and invalid inputs
- The AI module will be tested to ensure it generates valid and realistic schedules
- The database will be tested to ensure that data is stored and retrieved correctly

### Integration Testing:

All components will be tested together to ensure they communicate correctly:

- Frontend correctly sends requests to the backend API
- Backend correctly communicates with MongoDB and the Gemini API
- The complete flow from task input to schedule generation works end-to-end

### Security Testing

Authentication is properly enforced - unauthenticated users cannot access protected routes

- User data is only accessible to the authenticated owner
- The password lockout mechanism works correctly after 3 failed attempts

## 5.2 Evaluation:

Evaluation is the process of measuring whether the system has achieved its goals and meets the success criteria defined in Chapter 3.

### Functional Completeness

The system will be evaluated by checking that all four core features are fully implemented and working: AI-powered smart scheduling, dynamic re-scheduling, overload detection and alerts, and the performance analytics dashboard.

## **Accuracy Evaluation**

The accuracy of the AI scheduling module will be measured using the criteria defined in Section 3.3:

- At least 80% of generated study sessions must fit within the student's available time slots without conflicts
- At least 80% of students who test the system must report that the generated schedule feels realistic and manageable

## **Usability Evaluation**

A group of real students will be asked to use the system and complete a short questionnaire evaluating:

- Ease of use: How easy is it to add tasks and view the schedule
- Usefulness: Does the system help them plan their time better
- Satisfaction: Would they recommend the system to other students

## **Performance Evaluation:**

The system will be evaluated for response time and reliability:

- Schedule generation must complete within a few seconds
- The system must handle multiple users simultaneously without degradation
- No critical bugs or crashes must occur during normal use

## **Relevance Evaluation:**

The system will be evaluated against the core problem defined in Chapter 1 — does it genuinely help students reduce stress and improve their time management? This will be assessed through the student questionnaire and by comparing task completion rates before and after using the system.

## 6. References

- [illegible]